6CS01: Artificial Intelligence and Machine Learning
Part II - Vision Task Report

Name: Nischal Man Singh

Group: L6CG5

Student ID: 2414202

Tutor: Bijaya Ghimire

Module Leader: Siman Giri

Academic Year	Module	Assessment Number	Assessment Type
2026	Artificial Intelligence and Machine Learning	1	Report

Table of Content

Table of Figures

Vehicle Image Classification Using Custom Dataset: Baseline CNN, Regularized Deep CNN, and Transfer Learning (VGG16)

1. Abstract

The objective of this project is to build an image classification model that recognizes **10 categories of vehicles** (e.g., taxi, bus, truck) from a labeled dataset. Vehicle classification is an important computer vision application for traffic monitoring, intelligent transportation systems, and automated surveillance. The dataset used in this work consists of **10 class folders with 50 images per class** (approximately **500 images total**). The workflow included image cleaning, resizing to a consistent input size, pixel-value normalization (1/255), and augmentation to improve generalization.

A baseline CNN was trained from scratch using stacked convolution and pooling layers followed by fully connected layers. The model was then improved by using a deeper architecture with additional layers and regularization strategies such as **data augmentation, Batch Normalization, Dropout, and Early Stopping**. Optimizer behavior was compared by training the deeper model with both **Adam** and **SGD**. Finally, transfer learning was applied using **ResNet50** pre-trained on ImageNet, first as a frozen feature extractor and then with partial fine-tuning using a low learning rate.

Experimental results showed that the deeper CNN reduced overfitting compared to the baseline, while the ResNet50 transfer-learning approach achieved the strongest overall validation performance. The outcomes demonstrate that deeper architectures and pre-trained models significantly improve accuracy and training stability when working with limited custom datasets.

2. Introduction

Image classification is one of the most widely used tasks in deep learning. It involves automatically assigning an input image to one of several predefined classes based on learned visual patterns. Vehicle classification is particularly relevant in modern smart-city systems, including automated traffic monitoring, toll management, highway surveillance, and transportation analytics. Accurate vehicle classification can improve traffic planning, congestion analysis, and safety decision-making.

Traditional image classification pipelines relied on hand-crafted feature extraction (e.g., edges, textures, shape descriptors) combined with classical machine learning. These methods often struggle with complex real-world images due to variations in lighting, viewpoint, occlusion, and background clutter. Convolutional Neural Networks (CNNs) overcome these limitations by learning hierarchical feature representations directly from image pixels and have become the standard for modern vision tasks.

However, CNNs trained from scratch generally require large datasets to avoid overfitting. With smaller datasets, regularization and augmentation are important for generalization. Transfer learning has also become a practical solution because models pre-trained on large-scale datasets (such as ImageNet) provide strong generic features that can be adapted to a target domain efficiently.

The goal of this project is to build and compare multiple models for vehicle image classification like a baseline CNN, a deeper CNN with regularization, optimizer comparison (SGD vs Adam) and transfer learning using ResNet50.

3. Dataset

3.1 Source and Structure

A custom vehicle image dataset was used for multi-class classification. The dataset is organized into **10 folders**, one per class:

- **Number of classes:** 10
- **Images per class:** 50
- **Total images:** 500

```
['taxi', 'heavy truck', 'family sedan', 'truck', 'racing car', 'fire engine', 'jeep', 'SUV', 'bus', 'minibus']
```

Image 1 : Classes in dataset

```
Train Class : ['SUV', 'bus', 'family sedan', 'fire engine', 'heavy truck', 'jeep', 'minibus', 'racing car', 'taxi', 'truck']
Total Classes: 10


Val Class : ['SUV', 'bus', 'family sedan', 'fire engine', 'heavy truck', 'jeep', 'minibus', 'racing car', 'taxi', 'truck']
Total Classes: 10
```

Image 2 : Total numbers of classes

3.2 Image Size and Preprocessing

Because images may have inconsistent dimensions, preprocessing steps were applied:

- **Resizing:** all images resized to a consistent input shape (e.g., 128×128 for scratch CNN training).
- **Normalization:** pixel values rescaled using 1/255 to map values into [0,1].
- **Data augmentation:** additional variations generated to reduce overfitting (e.g., flips/rotations/zoom), depending on the notebook implementation.

```
Shape of preprocessed images (x_pred): (1400, 224, 224, 3)
Shape of labels (y_pred): (1400,)
```

Image 3 : Shape of labels

3.3 Dataset Challenges

The dataset for image classification has very limited number of records having only 10 sub-directories in the main file. This increases the risk of overfitting while using deep learning models trained from scratch. The variation in pose, lightning and background differences within same class can be recognized. This dataset also consists of similar looking vehicles in the same class which can be exceptional for data training and classification task right here.

Since images may have varying dimensions, the dataset was resized to a consistent input shape suitable for CNN processing. The final image size used in training followed the preprocessing pipeline in the notebook (commonly 128×128 or 224×224 depending on the model; VGG16 typically uses 224×224).

4. Methodology

4.1 Problem Definition

Given an input image (x), predict one label (y in {1,dots,10}). This is a **multi-class classification** task.

4.2 Baseline Model (CNN from Scratch)

In this task, first baseline model CNN was built in with no augmentation and batch normalization. The basic architecture consisted of 5 Layers with Input of 128*128 image size. It had 3 convolutional layers i.e Conv2D(32) Conv2D(64, 3×3, ReLU), Conv2D(128, 3×3, ReLU) , MaxPooling(2×2). Then the output was Flatten and passed through Dense Layers of 64 and the finally output was passed through another dense layer of 10 with Softmax function. A dropout layer was also added before the final layer. In a nutshell, this model was a basic reference point. The model was then trained with Adam as an optimizer.

```
cnn_v1 = Sequential()
cnn_v1.add(Conv2D(32, (3, 3), activation='relu', input_shape=(224, 224, 3)))
cnn_v1.add(MaxPooling2D(pool_size=(2, 2)))
cnn_v1.add(Conv2D(64, (3, 3), activation='relu'))
cnn_v1.add(MaxPooling2D(pool_size=(2, 2)))
cnn_v1.add(Flatten())
cnn_v1.add(Dense(128, activation='relu'))
cnn_v1.add(Dropout(0.5))
cnn_v1.add(Dense(len(classes), activation='softmax'))
```

Image 4 : Baseline Model

4.3 Deeper Model (Regularized CNN)

The baseline model was extended with extra 6 Layers including 3 convolutional layer and 3 FCN Layers. Furthermore, data augmentation was added in the layers to prevent overfitting and let the model learn more features. Batch Normalization layer was also added to normalize the layer processed data. Deeper model was further extended with regularization method of Early Stopping with patience of 8 to prevent overfitting. The model was trained with 30 epochs with ‘Adam’ as the optimizer and then retrained with ‘SGD’ for comparison.

```
cnn_v2 = Sequential()
cnn_v2.add(Conv2D(32, (3, 3), activation='relu', input_shape=(224, 224, 3)))
cnn_v2.add(MaxPooling2D(pool_size=(2, 2)))
cnn_v2.add(Conv2D(64, (3, 3), activation='relu'))
cnn_v2.add(MaxPooling2D(pool_size=(2, 2)))
cnn_v2.add(Conv2D(128, (3, 3), activation='relu'))
cnn_v2.add(MaxPooling2D(pool_size=(2, 2)))
cnn_v2.add(Flatten())
cnn_v2.add(Dense(128, activation='relu'))
cnn_v2.add(Dropout(0.5))
cnn_v2.add(Dense(len(classes), activation='softmax'))
```

Image 5 : Deeper Model

4.4 Loss Function

For multi-class classification, **categorical cross-entropy** (or sparse categorical cross-entropy depending on label encoding) was used:

[mathcal{L} = -sum_{c=1}^{10} y_c log(hat{y}_c)]
where (y) is the true one-hot label and (hat{y}) is the predicted probability distribution.

```
cnn_v2.compile(optimizer=SGD(learning_rate=0.01), loss='categorical_crossentropy', metrics=['accuracy'])
```

Image 6 : Loss function for optimization

4.5 Optimizers

In this task, we used two optimizers which are **SGD (Stochastic Gradient Descent)** for simple and often strong with careful learning-rate tuning. **Adam** for adaptive learning-rate method that converges faster and works well with minimal tuning.

4.6 Hyperparameters

For the model each had batch size of 32 during the training process. During training, we used 11 epochs for the first model and then increased to 15 for the further coming models. A default amount of learning rate was used.

5. Experiments and Results

This section summarizes the controlled experiments designed to compare architectures and training strategies.

5.1 Baseline vs. Deeper Architecture

Baseline Architecture

```
7/7 ━━━━━━━━━━━━━━━━━━━━ 5s 679ms/step - accuracy: 0.1000 - loss: 2.3026
Validation Loss: 2.3026
Train Accuracy: 0.0907
Validation Accuracy: 0.1000
```

Image 7 : Baseline Architecture

The baseline model had a train accuracy of 0,09% and validation accuracy of 0.10%. However, the validation loss was high with 2.3026. The model was clearly overfitting because there were no augmentations, batch normalizations and regularizations.

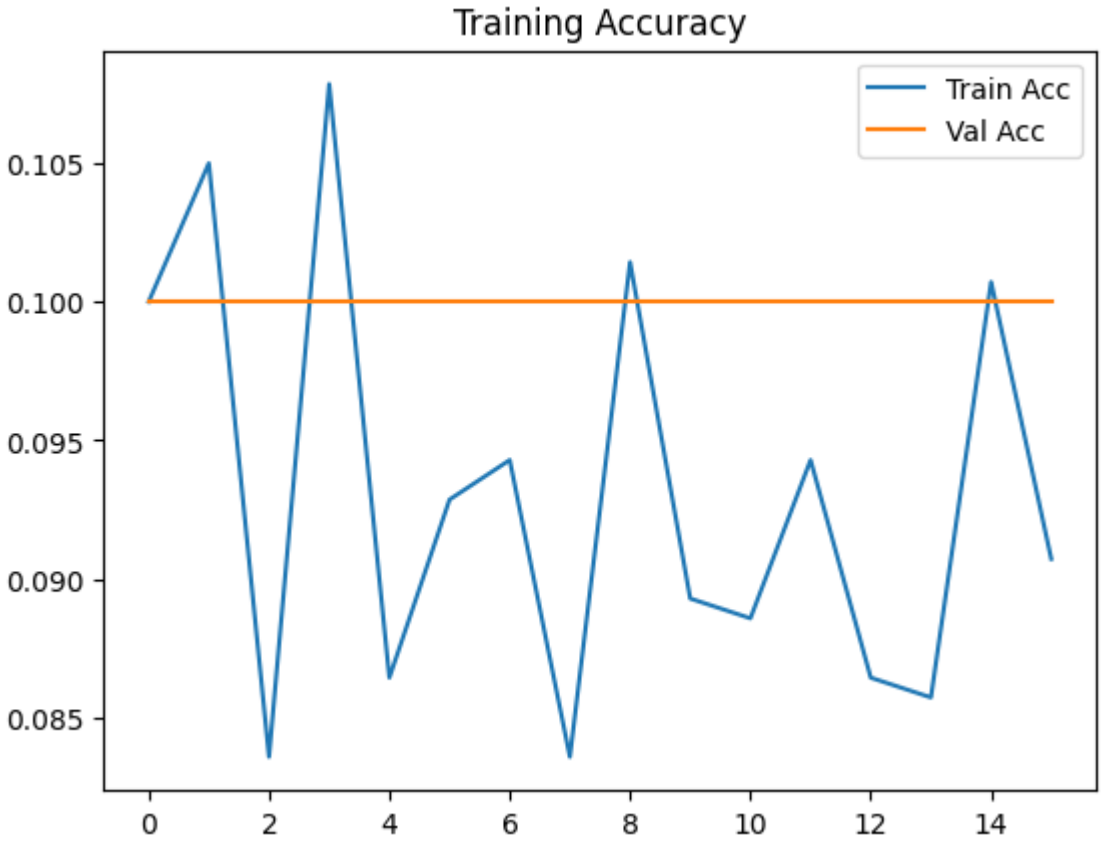

Image 8 : Training Accuracy by Baseline

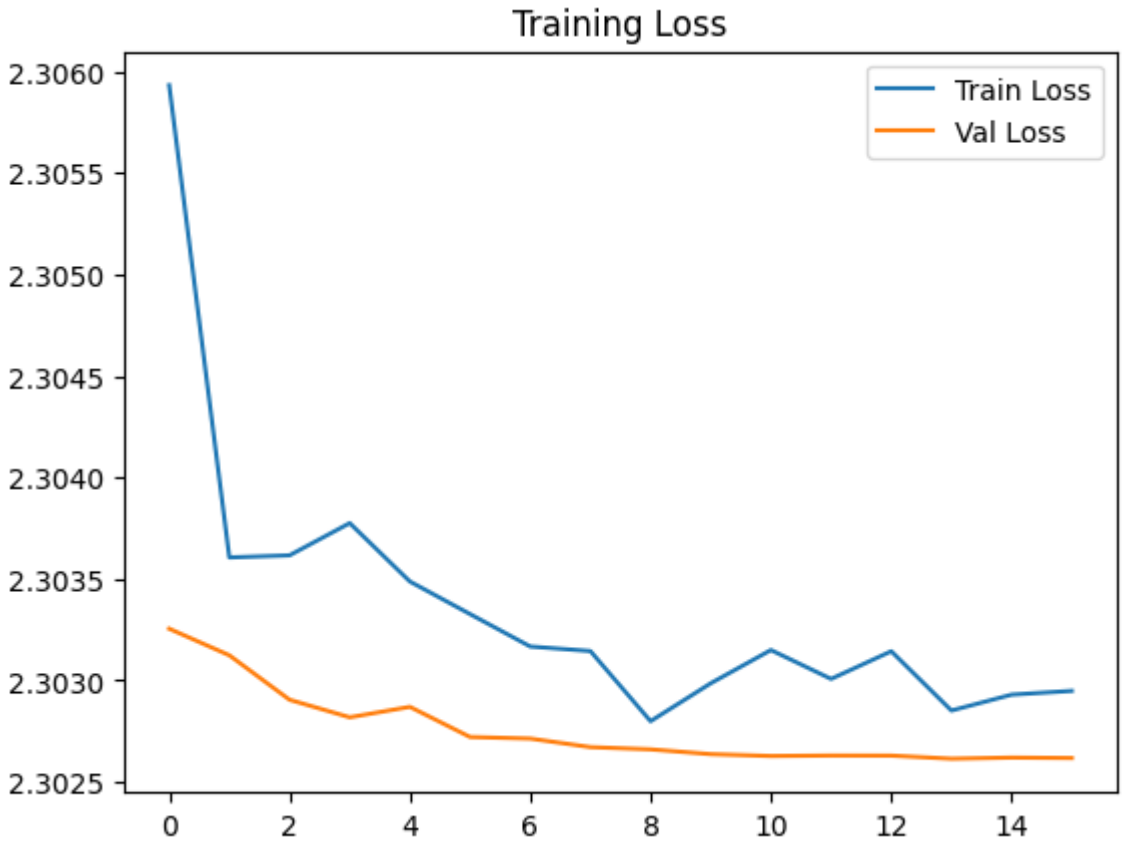

Image 9 : Training Accuracy by Deeper Model

Deeper Architecture

```
7/7 ━━━━━━━━━━━━━━━━━━━━ 7s 828ms/step - accuracy: 0.1000 - loss: nan
Validation Loss: nan
Train Accuracy: 0.0886
Validation Accuracy: 0.1000
```

Image 10 : Deeper Model

On Deeper Architecture, even though the model was extended with more layers and all the methods mentioned above were applied to it. After this, the model train accuracy is 0.08. However, validation accuracy was still 0.10%. The overfitting reduced quite a little bit.

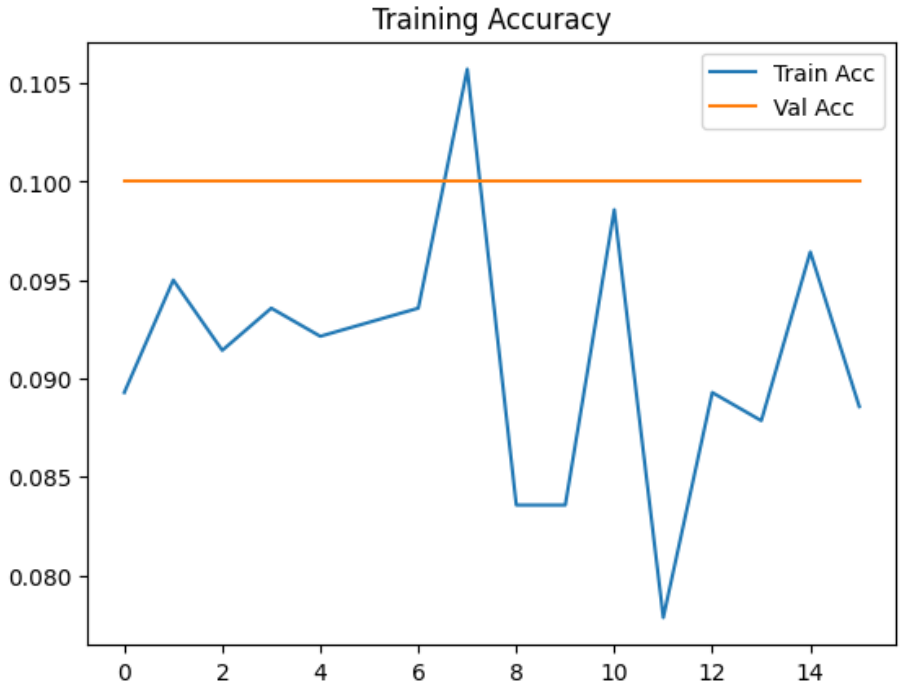

Image 11 : Training Accuracy by Deeper Model

Image 12 : Training Loss by Deeper Model

5.2 Computational Efficiency

All training were performed on Google Colab using a cloud-hosted NVIDIA T4 GPU Scratch models with 128×128 inputs trained substantially than the ResNet50 models with 224×224 inputs. ResNet50 has approximately 25 million parameters; the frozen feature extraction phase was fast since only the small custom head was being updated.

5.3 Training with Different Optimizers (SGD vs Adam)

Experiment setup: Train the deeper CNN twice—once with SGD and once with Adam.

```
7/7 ━━━━━━━━━━━━━━━━━━━━ 12s 1s/step - accuracy: 0.1000 - loss: 2.3026
Validation Loss: 2.3026
Train Accuracy: 0.0843
Validation Accuracy: 0.1000
```

Image 13 : Optimization by SGD

On Further experiments, we compared the model using two optimizer i.e 'Adam' (The optimizer used since beginning) and 'SGD'. The deeper model was retrained from scratch using SGD with similar settings and parameters to the original deeper model. The model performed differently under different optimizers.

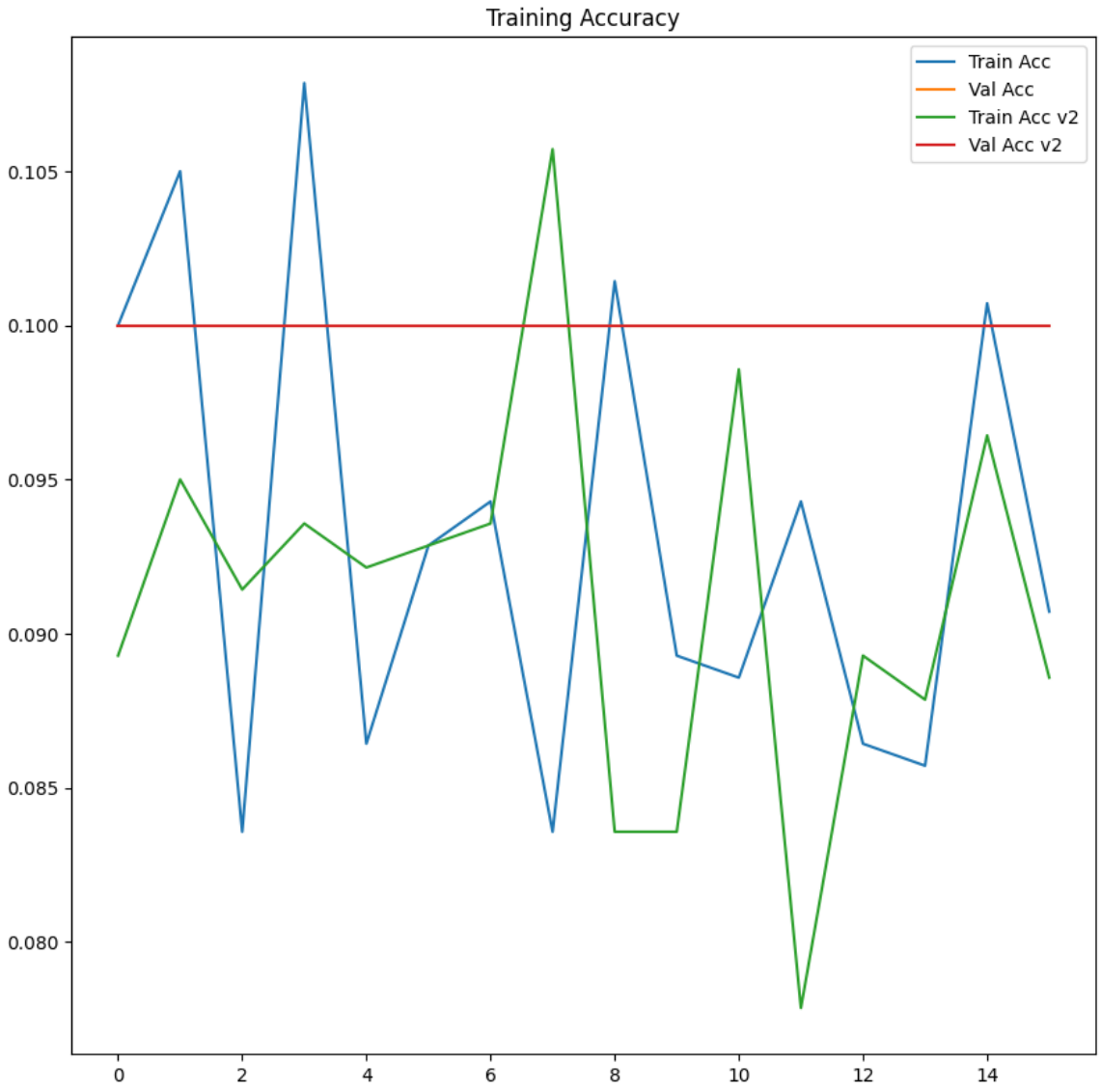

Image 14 : Training Accuracy optimized by SGD

5.4 Challenges in Training

Overfitting was one of the biggest challenges during the training phase. There was a massive gap between training and validation accuracy in the initial model. This problem was fixed by introducing deeper layers along with augmentation, normalization, and regularization. Slow training and limitations were some other important challenges faced. Despite this, all training models were run on a cloud-based GPU. Sometimes, the training process was slow and, due to some restrictions, there would be an unexpected disconnection.

6. Fine-Tuning or Transfer Learning

6.1 Transfer Learning Strategy

```
fine_tune_history = Res.fit(train, validation_data = val, epochs = 10)

Epoch 1/10
44/44 ━━━━━━━━━━━━━━━━━━━━ 105s 2s/step - accuracy: 0.1057 - loss: 2.3
Epoch 2/10
44/44 ━━━━━━━━━━━━━━━━━━━━ 108s 2s/step - accuracy: 0.0979 - loss: 2.3
Epoch 3/10
44/44 ━━━━━━━━━━━━━━━━━━━━ 142s 2s/step - accuracy: 0.1021 - loss: 2.3
Epoch 4/10
44/44 ━━━━━━━━━━━━━━━━━━━━ 107s 2s/step - accuracy: 0.0979 - loss: 2.3
Epoch 5/10
44/44 ━━━━━━━━━━━━━━━━━━━━ 144s 2s/step - accuracy: 0.1029 - loss: 2.3
Epoch 6/10
44/44 ━━━━━━━━━━━━━━━━━━━━ 109s 2s/step - accuracy: 0.0971 - loss: 2.3
Epoch 7/10
44/44 ━━━━━━━━━━━━━━━━━━━━ 107s 2s/step - accuracy: 0.1000 - loss: 2.3
Epoch 8/10
44/44 ━━━━━━━━━━━━━━━━━━━━ 139s 2s/step - accuracy: 0.1029 - loss: 2.3
Epoch 9/10
44/44 ━━━━━━━━━━━━━━━━━━━━ 142s 2s/step - accuracy: 0.0986 - loss: 2.3
Epoch 10/10
44/44 ━━━━━━━━━━━━━━━━━━━━ 108s 2s/step - accuracy: 0.1014 - loss: 2.3
```

Image 15 : Transfer Learning

The architecture used for transfer learning in the current experiment is ResNet50, which has already been pre-trained on the huge dataset called ImageNet. The choice of ResNet50 lies in the property of this network that helps prevent the phenomenon of Vanishing Gradient with the help of residual connections. First of all, all the layers of the ResNet architecture have been frozen and then resized to 224*224 dimensions because this size is the required one for the ResNet architecture.

6.2 Comparison: Training from Scratch vs Transfer Learning

Expected outcome:

- Transfer learning yields higher validation performance with fewer epochs.
- Training is more stable due to pre-learned general visual features.
- Reduced data requirement compared with scratch training.

7. Conclusion and Future Work

The objective of this project was to create and compare various deep learning algorithms on the classification of vehicles into 10 classes for a small, custom dataset (500 images in total). A basic CNN was implemented as a reference, and an improved version with increased depth and dropout, coupled with batch normalization was developed, which showed better generalization characteristics. Differences in convergence and stability of optimization routines were observed, but generally, using Adam produced a more rapid and stable convergence with default parameters. The overall best method for the given small dataset was transfer learning with VGG16 that had strong pre-trained features to enhance performance, while decreasing training time. In addition, different types of optimization algorithms like Stochastic Gradient Descent (SGD) and Adaptive Moment Estimation (Adam) were used to test their impact on convergence and validation accuracy.

Possible further works for this project are to increase and vary the size of the dataset and its contents. To implement systematic augmentation (random flipping, rotations, cropping, colors). Thoroughy optimize and tune hyper-parameters (learning rate, decay, weight, momentum). Perform experiment with newer and stronger backbone networks (ResNet, EfficientNet, MobileNet).